

Week 5: Creating Interactive Applications

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will be able to write a **stateful widget** and update the screen with **setState**; explain the **life cycle** of a **State** object (**initState**, **build**, **dispose**, ...); take input from the user with a **TextField** and react to it with **onChanged** and **onSubmitted**; build a small interactive application (a text that follows what you type, and a sum of two numbers); spread an app over **several files**; write your own widget that receives a **callback function**; use an **ElevatedButton** and the other button widgets; build a complete app, a **stopwatch**, driven by a **Timer** and its own buttons; and show a **scrolling list** with **ListView**.

Every step of the lecture is a complete Flutter app. The files are in the `lib` folder of the project `flutter_examples`, inside `code/week05`. The code is not printed in full here: each section quotes only the few lines that matter, taken directly from those files, and automated tests check that each app does what is described. Open the file to read the rest. See the guide *Running the Flutter apps in VS Code* to run them.

1 What are interactive applications?

- An **interactive application** lets the user interact with the app *while it runs*: the user types, taps, and the screen changes in response.
- In Flutter we build interactive applications with **stateful widgets**, the `StatefulWidget` class of week 4.
- This week we learn to take input from the user with the `TextField` widget.
- We also learn to **distribute our classes across several libraries** (files), as every real application does.

2 The stateful widget

In week 4 we saw that a stateless widget never changes, and that a stateful widget can change and be drawn again. Since our apps now react to the user, we need stateful widgets. A stateful widget is made of **two classes**:

Class	Role
The widget class (extends StatefulWidget)	is <i>immutable</i> : it holds nothing that changes. It has a single method, <code>createState()</code> , which returns the <code>State</code> object.
The state class (extends State<...>)	keeps the <i>data that changes</i> (its fields), and has the <code>build</code> method. It redraws the widget when the data changes.

The structure is always the same. The name of the state class starts with an underscore: it is private to the file (week 3). In Android Studio type `stf` and choose `stful`; in VS Code type `stful`: the editor writes both classes for you.

```

1 // 1. The widget class: immutable, it only creates its State.
2 class CounterPage extends StatefulWidget {
3   const CounterPage({super.key});
4
5   @override
6   State<CounterPage> createState() => _CounterPageState();
7 }
8
9 // 2. The State class: it keeps the data that changes and builds the screen.
10 class _CounterPageState extends State<CounterPage> {
11   int _count = 0;
12
13   void _increment() {
14     // setState runs the function, then Flutter calls build again.
15     setState(() {
16       _count++;
17     });
18   }

```

setState

We tell Flutter that the data changed by calling `setState`. It takes a function; Flutter runs it, then calls `build` again, and only the parts of the screen that differ are redrawn.

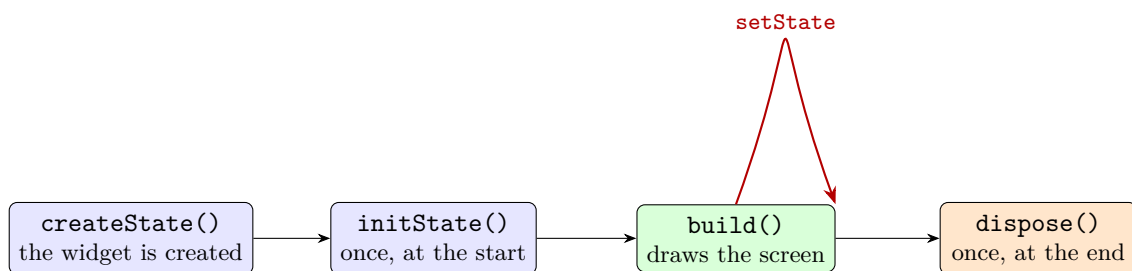


Figure 1: The life of a State object. `build` runs at the start and again after each `setState`. Most widgets only override `build`; we use `initState` and `dispose` in section 9 and again in section 11.

- Put in `setState` the lines that *change the data*; the drawing is done by `build`.
- `build` may be called at any moment and many times: it must be fast, and free of side effects.
- The data lives in the *state* object, which stays alive while the widget is on the screen; the widget object itself may be created again and again.

What happens when you call setState

`setState` does not draw anything itself. It *marks* the widget as **dirty**: “this one must be built again”. On the next frame, Flutter’s framework rebuilds every dirty widget of the tree: it calls their `build` method, compares the result with what is on the screen, and updates only what changed.

- **Never call build yourself.** Flutter calls it, at the right moment; you only call `setState`.
- All widgets, stateless or stateful, are *immutable*. What can change in a stateful widget is its *State* object.

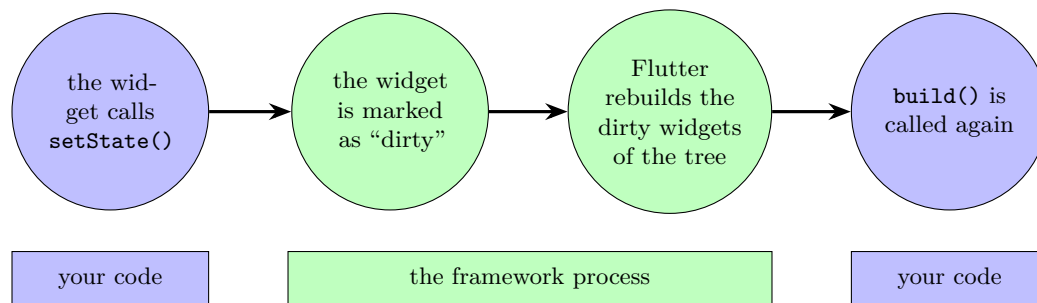


Figure 2: How `setState` and `build` work together in Flutter's render loop.

- The function that you give to `setState` is optional: `_count++; setState(() {});` works exactly like `setState(() { _count++; });`. We put the change inside because it is clearer: the reader sees what changed.
- Keep the function short, a simple assignment. A slow or complex operation inside it makes the screen slow, because the next frame waits for it.

The life cycle methods

Unlike a `StatelessWidget`, which has only `build`, a `State` object has a **life cycle**: Flutter calls its methods in a fixed order. You override the ones you need. Most of the time these are `initState`, `build` and `dispose`.

Method	Called	You use it to
<code>initState</code>	once, after the constructor, before the widget is in the tree	initialise the fields that are not final : start a <code>Timer</code> , create a controller. It does the job of a constructor. Do not read <code>Theme.of(context)</code> here
<code>didChangeDependencies</code>	right after <code>initState</code> , and again when something the widget depends on changes (<code>Theme</code> , <code>MediaQuery</code> , a provider)	set up what needs the <code>BuildContext</code> and inherited data
<code>didUpdateWidget</code>	when the parent is built again and gives the same <code>State</code> a new widget (new parameters)	compare <code>oldWidget</code> with <code>widget</code> and react to a change
<code>build</code>	required: after each method above and after each <code>setState</code>	describe the interface: return the widgets. This is where you create the UI
<code>reassemble</code>	only in development, at a hot reload	rarely needed
<code>deactivate</code>	when the <code>State</code> is removed from the tree	rarely needed
<code>dispose</code>	when the <code>State</code> is destroyed for good	release resources : cancel <code>Timers</code> , dispose controllers, close streams

In short: at the start, `initState`, `didChangeDependencies`, `build`; at each change of parameters, `didUpdateWidget`, `build`; at each `setState`, `build`; at the end, `deactivate`, `dispose`. Example `09_lifecycle_log` makes a child widget write its own name in a log each time one of its methods

is called: change its parameter, hide it and show it again, and read the order on the screen. Press **r** (hot reload) while it runs: **reassemble** appears.

Forgetting **dispose** for a long-running resource, such as a **Timer** or a stream, leads to memory leaks, unpredictable behaviour, and sometimes crashes: the timer keeps ticking after its widget has been destroyed.

Changing a variable is not enough

The screen shows what **build** returned the *last* time it ran. If you change a variable without **setState**, **build** does not run and the screen does not change. Example `01_setstate_needed` has two buttons that both add 1 to a variable:

```

1 void _addWithSetState() {
2   setState(() {
3     _value++;
4   });
5 }
6
7 void _addWithoutSetState() {
8   _value++; // the variable changes, but build is NOT called again
9 }
```

Press the second button three times: nothing changes on the screen. Then press the first one: the screen jumps to 4, because **build** finally runs and reads the current value. **Rule:** whenever the data shown on the screen changes, change it in **setState**.

3 The TextField widget

The **TextField** widget takes input from the user. It has many fields; these are the main ones:

Field	Type	Meaning
style	TextStyle	the font of the typed text: size, color, ...
decoration	InputDecoration	what surrounds the text: border, hint, label, icon (below)
onChanged	function(String)	called <i>every time</i> the text changes, with the new text
onSubmitted	function(String)	called <i>once</i> , when the user finishes editing, with the text
keyboardType	TextInputType	which keyboard a phone shows: text, number, email, ...
controller	TextEditingController	lets the program read and change the text (section 11)
obscureText	bool	true hides the letters, for a password
maxLength	int	the maximum number of characters

The slides call the second field **decorations**. The real name is **decoration**, in the singular.

The **InputDecoration** has its own fields:

Field of InputDecoration	Meaning
border	the frame; <code>OutlineInputBorder()</code> draws a rectangle around the field
hintText	grey text shown while the field is empty
labelText	a label that moves above the text when you type
prefixIcon, suffixIcon	an icon inside the field, at the left or at the right
helperText	a small help text under the field
errorText	a red message under the field; <code>null</code> (the default) shows nothing

onChanged and onSubmitted

Both fields are **anonymous functions** with one parameter, the text (week 2). They are called at different moments: `onChanged` after each letter, `onSubmitted` once, when the user presses Enter on a computer or the check mark (the “done” key) of the phone keyboard. Example 02_textfield_events shows the two side by side:

```

1 child: TextField(
2   style: const TextStyle(fontSize: 18),
3   decoration: const InputDecoration(
4     border: OutlineInputBorder(),
5     labelText: 'Name',
6     hintText: 'Type your name',
7     prefixIcon: Icon(Icons.person),
8   ),
9   onChanged: (text) {
10     setState(() {
11       _changed = text;
12     });
13   },
14   onSubmitted: (text) {
15     setState(() {
16       _submitted = text;
17     });
18   },
19 ),

```

A `TextField` must be inside a `Scaffold` (or another `Material` widget), and it needs a width: in a `Row`, or when we want a fixed size, we put it in a `SizedBox` or an `Expanded`.

4 Our first interactive application

The first application has a `Text` and a `TextField`. The text that the user types in the `TextField` is displayed at once in the `Text`.

Create a new Flutter project as in week 4 (`flutter create week5`, or **File** → **New Flutter Project** in Android Studio). We write the app in small steps. To keep every step easy to read, example 03_you_typed has *all* the classes in one file; in section 5 we spread them over three files, as in the slides.

Step 1: main.dart

`main` runs the app. `MyApp` is stateless: it only creates the `MaterialApp` whose `home` is our stateful widget `Home`. In the two-file version, `Home` is defined in a file `home.dart`, so `main.dart` imports it with `import 'home.dart';`.

```

1 void main() => runApp(const MyApp());

```

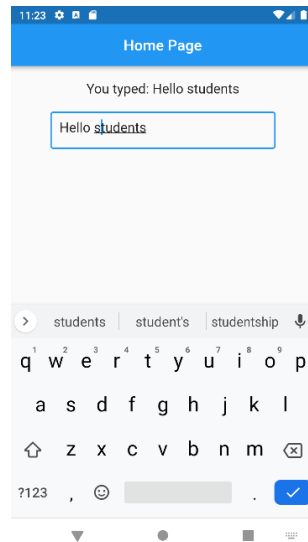


Figure 3: The first application: the text follows what is typed.

```

2
3 class MyApp extends StatelessWidget {
4   const MyApp({super.key});
5
6   @override
7   Widget build(BuildContext context) {
8     return const MaterialApp(
9       title: 'CSCI410 Week 5',
10      debugShowCheckedModeBanner: false,
11      home: Home(),
12    );
13  }
14 }

```

Step 2: the Home widget

The stateful widget Home has the two classes of section 2. The build method of the state class returns a Scaffold with an AppBar whose title is centered:

```

1 return Scaffold(
2   appBar: AppBar(
3     title: const Text('Home Page'),
4     centerTitle: true,
5   ),
6   body: ...

```

Step 3: the Text and the TextField

The body is a Center that holds a Column. In the Column, a Text shows the variable `_text`, and a TextField lets the user type.

```

1 const SizedBox(height: 20.0),
2 Text('You typed: $_text', style: const TextStyle(fontSize: 18.0)),
3 const SizedBox(height: 20.0),
4 SizedBox(
5   width: 300.0,
6   height: 50.0,
7   child: TextField(
8     style: const TextStyle(fontSize: 18.0),

```

```

9      decoration: const InputDecoration(
10        border: OutlineInputBorder(),
11        hintText: 'Enter some text',
12      ),
13      onChanged: (text) {
14        updateText(text); // call the updateText method
15      },
16    ),
17  ),

```

- We put the `TextField` inside a `SizedBox` to give it a **width and a height** (300 by 50 logical pixels).
- We gave the `TextField` a border with the `InputDecoration` widget, and a font size with the `TextStyle` widget.
- The `onChanged` function is called every time the text of the `TextField` changes. It calls `updateText`, which updates the text of the `Text` widget.
- In 'You typed: \$_text', the \$ puts the value of the variable in the string (string interpolation, week 1).

Step 4: the `updateText` method

The state class has a field that holds the text of the `TextField`, and the method `updateText`. It must call `setState`, so that the state class redraws the widget.

```

1 class _HomeState extends State<Home> {
2   // A field to hold the TextField's text.
3   String _text = '';
4
5   // Called when the text of the TextField changes. We need setState so that
6   // the State class redraws the widget.
7   void updateText(String text) {
8     setState() {
9       _text = text;
10    };
11  }

```

The complete flow, at each letter typed:

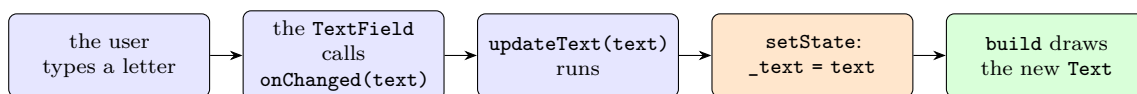


Figure 4: From the keyboard to the screen.

5 Several files: libraries and imports

A real application is not written in one file. In Dart, **every .dart file is a library**, and we distribute the classes of an app across several libraries. The sum application of this lecture (example 05_sum_app) has three:

- `import 'home.dart';` makes the classes of another file usable. A path that does not start with `package:` is **relative to the current file**. The files of the Flutter SDK, and of any package, are imported with `package:`, for example `package:flutter/material.dart`.
- Each file imports what *it* uses: `home.dart` imports `my_text_field.dart`, and `main.dart` imports `home.dart`. `main.dart` does not need the text field.

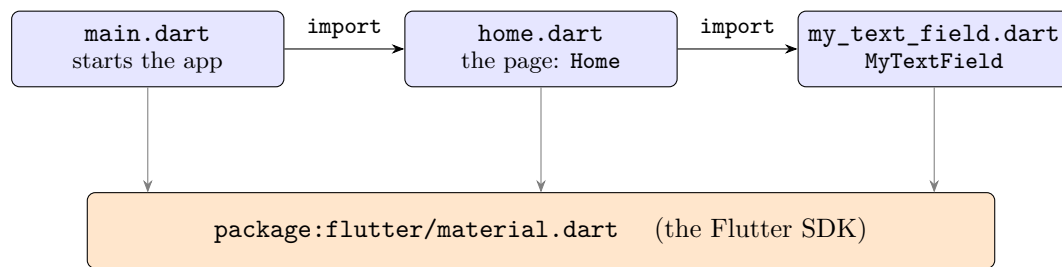


Figure 5: The three libraries of the sum app. An arrow means “imports”. Each file also imports the Flutter widgets.

- **Privacy is per library, not per class** (week 3): a name that starts with an underscore can be used only inside its own file. That is why `_HomeState` can be hidden from `main.dart`, which only needs `Home`.
- Good habits: a file name in `snake_case` (`my_text_field.dart`); one main class per file; `main.dart` only starts the app; related files in a folder.
- You can import only some names with `show` (`import 'a.dart' show A;`), leave some out with `hide`, or give a prefix with `as` (`import 'a.dart' as a;` used as `a.A`). Example `main.dart` of the project uses `as`, to start the sum app.

6 A custom TextField widget and callbacks

We will use text fields with the same design many times: same size, same border, same font. Instead of copying the code, we create **our own widget**, `MyTextField`. Then the design is written once, and a change made in one place changes every field.

```

1 class MyTextField extends StatelessWidget {
2   const MyTextField({
3     super.key,
4     required this.hint,
5     required this.onChanged,
6   });
7
8   final String hint; // the hintText of the TextField
9   final ValueChanged<String> onChanged; // the callback function
10
11   @override
12   Widget build(BuildContext context) {
13     return SizedBox(
14       width: 300.0,
15       height: 50.0,
16       child: TextField(
17         style: const TextStyle(fontSize: 18.0),
18         decoration: InputDecoration(
19           border: const OutlineInputBorder(),
20           hintText: hint,
21         ),
22         onChanged: onChanged, // call the callback with the new text
23       ),
24     );
25   }
26 }
  
```

- `MyTextField` is **stateless**: it keeps no data of its own. It receives, when it is created, the `hint` to show and a **function** to call: `onChanged`.
- Both are **final** fields, and **required** named parameters of the constructor (week 2).

- The type `ValueChanged<String>` is the name Flutter gives to “a function that takes a `String`”. The slides write `Function(String)`; both work, but the named type says more.

Callback functions

A function that we *give* to another piece of code, to be called *later*, is a **callback**. Here the page gives `MyTextField` its own method `updateText`:

```
1 MyTextField(onChanged: updateText, hint: 'Enter some text'),
```

When the user types, the `TextField` calls `onChanged`, which is `updateText` of the page, which calls `setState`. The widget that owns the data (the page) stays in charge of it; the field only reports what happens.

There are no parentheses after `updateText`. `onChanged: updateText` gives the function (functions are objects, week 2). `onChanged: updateText()` would call it at once, while the page is being built.

The function types that you will meet most often:

Type	Means	Used by
<code>VoidCallback</code>	a function with no parameter that returns nothing	<code>onPressed</code> of a button
<code>ValueChanged<T></code>	a function that takes one value of type <code>T</code>	<code>onChanged</code> , <code>onSubmitted</code> of a <code>TextField</code>

7 The sum application

The second application has two `TextFields` and a button. The user enters *X* and *Y* and presses **SUM**: the application displays their sum in a `Text`. It is the example `05_sum_app`, and the app that `flutter run` starts.

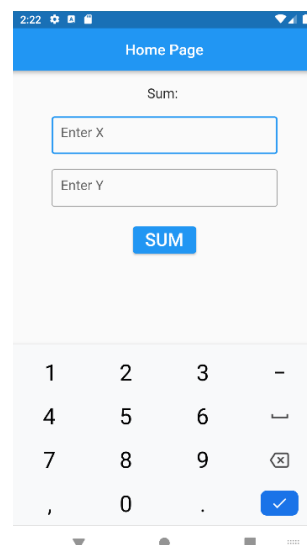


Figure 6: The sum application, with the numeric keyboard.

The state class

The state class holds the two numbers and the text to show. `updateX` and `updateY` are the callbacks of the two fields; they only *store* the number. The button calls `updateText`, which computes the sum inside `setState`.

```

1 class _HomeState extends State<Home> {
2   String _text = '';
3
4   // The numbers typed by the user. null means: empty, or not a number.
5   // (The slides use -1 for "empty"; null does not forbid the value -1.)
6   double? _x, _y;
7
8   // Called when the SUM button is pressed.
9   void updateText() {
10    setState(() {
11      final x = _x;
12      final y = _y;
13      if (x == null || y == null) {
14        _text = 'Please fill all fields';
15      } else {
16        _text = (x + y).toString();
17      }
18    });
19  }
20
21  // Called each time the text of a field changes. They only store the number:
22  // the screen does not change until the button calls setState.
23  // double.tryParse returns null for a text that is not a number, where
24  // double.parse would stop the app with a FormatException.
25  void updateX(String x) {
26    _x = double.tryParse(x.trim());
27  }
28
29  void updateY(String y) {
30    _y = double.tryParse(y.trim());
31  }

```

Notice that `updateX` and `updateY` do not call `setState`: nothing on the screen depends on the numbers until the button is pressed, so there is nothing to redraw.

The body

The body of the Scaffold is the previous Column, with the Text that shows 'Sum: \$_text', two MyTextField widgets with the callbacks `updateX` and `updateY`, and the button:

```

1 ElevatedButton(
2   onPressed: () {
3     updateText();
4   },
5   child: const Text('SUM', style: TextStyle(fontSize: 24.0)),
6 ),

```

The sum of two double values is a double, so $2.5 + 4$ is shown as 6.5, and $1 + 3$ as 4.0.

What we changed from the slides

The slides show the same structure. Our code differs in a few places, on purpose:

Slides	Our code	Why
<code>double.parse(x)</code>	<code>double.tryParse(x)</code>	<code>parse</code> stops the app with a <code>FormatException</code> if the text is not a number (for example <code>abc</code> , or <code>1.2.3</code>); <code>tryParse</code> returns <code>null</code>
<code>double _x = -1</code> means “empty”	<code>double? _x, null</code> means “empty”	with <code>-1</code> as marker, the user cannot add the number <code>-1</code>
<code>Function(String) f</code> , not <code>final</code>	<code>final ValueChanged<String> onChanged</code>	a widget is immutable; the name is the one of <code>TextField</code>
<code>Key? key</code> and <code>super(key: key)</code>	<code>super.key</code>	same meaning, shorter (Dart 2.17)
default keyboard	<code>keyboardType</code> for numbers	a phone shows the numeric keyboard of the screenshot

8 The ElevatedButton widget

`ElevatedButton` draws a button with a shade: it creates an increased shade effect when you press it.

Field	Meaning
<code>child</code>	what is inside the button: usually a <code>Text</code> , or an <code>Icon</code>
<code>onPressed</code>	an anonymous function, with no parameter, called when the button is pressed. If it is null, the button is disabled (grey) and does not react
<code>style</code>	the look of the button, built with <code>ElevatedButton.styleFrom(...)</code>

Other buttons, and two ways to style them

Buttons are among the most important pieces of an interface. They look different, but their functionality is the same: a widget that accepts a function, and runs it when it detects a touch, a press or a click. If `onPressed` is `null`, the button is disabled. The main buttons are:

Button	Look and use
<code>ElevatedButton</code>	filled, with a shade: the main action
<code>FilledButton</code>	filled, without a shade (Material 3)
<code>OutlinedButton</code>	a thin frame: a secondary action
<code>TextButton</code>	only text: a discreet action
<code>IconButton</code>	an icon that can be tapped: app bars, toolbars
<code>FloatingActionButton</code>	the round button that floats over the page
<code>DropDownButton</code>	shows a list of choices when tapped
<code>CupertinoButton</code>	the iOS style (week 4, part 3)

There are two ways to give a button its colors and shape. The simple one is `styleFrom`, used above. The general one is a `ButtonStyle`, whose fields are not values but `WidgetStateProperty` objects: a value *for each state* of the button (pressed, hovered, disabled, ...). `WidgetStateProperty.all(color)` uses the same value in every state:

```

1 ElevatedButton(
2   style: ButtonStyle(
3     backgroundColor: WidgetStateProperty.all<Color>(Colors.green),
4     foregroundColor: WidgetStateProperty.all<Color>(Colors.white),
5   ),
6   onPressed: _startTimer,
7   child: const Text('Start'),
8 )

```

- With `.all(color)` the *disabled* button keeps its green: nothing tells the user that it cannot be used. `styleFrom(backgroundColor: ...)` colors only the enabled button, and the disabled one keeps the grey look. We use `styleFrom` in the stopwatch.
- Older code and books write `MaterialStateProperty` (the old name of `WidgetStateProperty`), and `ElevatedButton.styleFrom(primary: ..., onPrimary: ...)` (now `backgroundColor` and `foregroundColor`). They no longer work in recent versions of Flutter.
- `onPressed` can be a method (`onPressed: _startTimer`) or a closure (`onPressed: () { ... }`). For one line a closure is fine; as soon as the button also decides whether it is active, with a ternary operator, a method is easier to read.

Changing the design: a round button

The slides leave as an exercise to make the button round. The `shape` field of `styleFrom` does it:

Value of shape	Result
<code>const StadiumBorder()</code>	a “pill”: the two ends are half circles
<code>RoundedRectangleBorder(borderRadius: BorderRadius.circular(12))</code>	rounded corners
<code>const CircleBorder()</code>	a circle (put an <code>Icon</code> in it, and some padding)

`styleFrom` has other useful fields: `backgroundColor`, `foregroundColor` (the color of the label), `elevation` (the size of the shade), `padding` and `minimumSize`. Example 07_button_styles shows all of them:

```

1 ElevatedButton(
2   onPressed: () => _press('PILL'),
3   style: ElevatedButton.styleFrom(
4     shape: const StadiumBorder(),
5     backgroundColor: Colors.deepPurple,
6     foregroundColor: Colors.white,
7     padding:
8       const EdgeInsets.symmetric(horizontal: 32, vertical: 14),
9   ),
10  child: const Text('PILL'),
11 )

```

The same example has a button whose `onPressed` is `null` until a `Switch` is turned on:

```

1 ElevatedButton(
2   onPressed: _lastEnabled ? () => _press('DISABLED') : null,
3   child: const Text('DISABLED'),
4 )

```

9 A complete example: the stopwatch

The apps so far changed when the *user* did something. To finish the lecture we build one app, in three steps, that also changes by itself as *time* passes: a **stopwatch** that counts, starts and stops, and remembers its laps. Each step is one file (examples 10_stopwatch_timer, 11_stopwatch_buttons and 12_stopwatch_laps). It uses everything of this week: a stateful widget, `setState`, the life cycle, buttons, and a scrolling list.

Step 1: state that changes by itself

A `Timer.periodic` calls a function again and again. In that function we change a counter inside `setState`, and Flutter draws the new text. The three life cycle methods each have a job:

- `initState` creates the timer, once, when the State object is created;
- `build` draws the number;
- `dispose` cancels the timer when the screen is closed.

```

1 class _StopWatchState extends State<StopWatch> {
2   int _seconds = 0;
3   late final Timer _timer;
4
5   @override
6   void initState() {
7     super.initState();
8     _timer = Timer.periodic(const Duration(seconds: 1), _onTick);
9   }
10
11   void _onTick(Timer time) {
12     if (mounted) {
13       setState(() {
14         ++_seconds;
15       });
16     }
17   }
18
19   String _secondsText() => _seconds == 1 ? 'second' : 'seconds';
20
21   @override
22   void dispose() {
23     _timer.cancel(); // stop ticking when the screen is closed
24     super.dispose();
25   }

```

- `if (mounted)`: a timer may tick after the widget has left the tree. `setState` would then stop the app with “`setState()` called after `dispose()`”. `mounted` is `false` in that case.
- `late final Timer _timer`: “I will give it a value before it is used”. It is safe here because `initState` always runs first.
- The text uses the singular: *1 second*, *2 seconds*.
- `Theme.of(context).textTheme.headlineSmall` is a ready-made text style of the theme (older code uses `headline5`).

Step 2: Start and Stop buttons

Now the user controls the timer. The state gets a `bool`, `_isTicking`, and the two buttons use it to say whether they are active: a ternary expression gives `null`, and `null` disables a button. Only one of the two buttons is active at any time:

```

1 ElevatedButton(onPressed: _isTicking ? null : _startTimer, child: const Text('
  Start')),
2 TextButton(onPressed: _isTicking ? _stopTimer : null, child: const Text('Stop'))
  ,

```

Note that `_startTimer` has **no parentheses**: it is passed, not called. The two methods create and cancel the timer, and change `_isTicking` inside `setState`, which redraws both buttons:

```

1 class _StopWatchState extends State<StopWatch> {
2   int _seconds = 0;
3   bool _isTicking = false;
4   Timer? _timer; // null until Start is pressed
5
6   void _onTick(Timer time) {
7     if (mounted) {
8       setState(() {
9         ++_seconds;
10      });
11    }
12  }
13
14  void _startTimer() {
15    _timer = Timer.periodic(const Duration(seconds: 1), _onTick);
16    setState(() {
17      _seconds = 0;
18      _isTicking = true;
19    });
20  }
21
22  void _stopTimer() {
23    _timer?.cancel();
24    setState(() {
25      _isTicking = false;
26    });
27  }

```

The timer is now created by a button, so it may not exist when the screen closes. With late `Timer` timer and no press on Start, dispose would stop the app with a `LateInitializationError`. We declare it `Timer? _timer` and call `_timer?.cancel()`. The tests of the example check both cases.

Step 3: laps and a scrolling list

The stopwatch now counts in tenths of a second, and a **Lap** button stores the time of the lap that just finished in a list, and starts the next one:

```

1 void _lap() {
2   setState(() {
3     _laps.add(_milliseconds);
4     _milliseconds = 0;
5   });
6 }
7
8 String _secondsText(int milliseconds) {
9   final seconds = milliseconds / 1000;
10  return '${seconds.toStringAsFixed(1)} seconds';
11 }

```

The list of laps is shown in a `ListView` (next section). The screen is split into two equal halves, each in an `Expanded`: the blue counter panel on top, the laps below. As the build method

grows, we split it into small methods that return a `Widget`: `_buildCounter`, `_buildControls`, `_buildLapDisplay`.

```

1  @override
2  Widget build(BuildContext context) {
3    return Scaffold(
4      appBar: AppBar(title: const Text('Stopwatch')),
5      body: Column(
6        children: [
7          // Each Expanded takes half of the screen.
8          Expanded(child: _buildCounter(context)),
9          Expanded(child: _buildLapDisplay()),
10         ],
11       ),
12     );
13 }

```

- The **Lap** button is active only while the stopwatch runs; **Start** clears the laps.
- A **Timer** is not exact: adding 100 at each tick drifts after a long time. For a precise stopwatch, use Dart's `Stopwatch` class, which reads the clock, and let the **Timer** only redraw the screen.

10 Making it scroll

Nearly every app has scrolling content: when a list is taller than the screen, we need a scrollable widget. The simplest is `ListView`. Like a `Column`, it places its children one after another, but when there is no more room, a `Column` overflows and a `ListView` **scrolls**, driven by drag gestures.

```

1  Widget _buildLapDisplay() {
2    return ListView(
3      children: [
4        // collection-for: one ListTile for each lap
5        for (var i = 0; i < _laps.length; i++)
6          ListTile(
7            leading: Text('Lap ${i + 1}'),
8            title: Text(_secondsText(_laps[i])),
9          ),
10     ],
11   );
12 }

```

Here we use **collection-for** (a `for` inside a list, as in a `for ... in` loop) to create one `ListTile` for each lap. The list grows as laps are added.

Long lists: `ListView.builder`

`ListView(children: [...])` creates *all* the widgets at once: fine for a few items. For a long list, `ListView.builder` creates each row only when it is about to appear on the screen. You give it `itemCount` and an `itemBuilder(context, index)`. In example 13_scrolling the list has 1000 items, but only the twenty or so on the screen exist.

```

1  Expanded(
2    child: ListView.builder(
3      physics: _choices[_choice], // null = the platform's default
4      itemCount: 1000,
5      itemBuilder: (context, index) {
6        return ListTile(
7          leading: const Icon(Icons.label_outline),
8          title: Text('Item $index'),
9        );
10     },

```

```

11 ),
12 ),

```

Scroll physics: the feel of a scroll

Scrolling is **platform aware**. On iOS a list bounces when you reach its end; on Android it stops, with a glow; the web and the desktop feel different again. These behaviours are **ScrollPhysics**. A list picks the one of the platform by default, and the **physics** field lets you choose another:

physics	Behaviour
BouncingScrollPhysics()	bounces at the ends, as on iOS
ClampingScrollPhysics()	stops at the ends, as on Android
NeverScrollableScrollPhysics()	does not scroll with the finger at all

Example 13_scrolling lets you choose. Do not override the platform's behaviour without a good reason: users expect the feel of their own platform, and iOS habits on Android confuse them.

Constraints go down, sizes go up

A scrolling widget must know how much room it has. Flutter has a rule: **constraints go down, sizes go up**.

- A parent passes *constraints* to its children: a minimum and a maximum for the width and the height (a **BoxConstraints** object).
- A child chooses its size within them and passes it back *up* to the parent.

A **Column** lays its children out by their *own* size, and gives them unlimited height. This works for **Container**, buttons and **Text**, but not for a **ListView**, which wants as much height as it can get. In the stopwatch we wrapped it in an **Expanded**, which tells the **ListView** how much room it has. Try removing the **Expanded**: the list disappears and the Debug Console shows a long error, "Vertical viewport was given unbounded height". The explosion of lines has little to do with your code; it just means that you have an unbounded scrolling widget.

A scrolling widget inside a Row or a Column: always wrap it in Expanded or Flexible first.

Which scrolling widget?

Widget	Use it for
ListView	a short scrolling column (or row) of widgets
ListView.builder	a long or endless list: rows built only when visible
ListView.separated	a list with a divider between the rows
GridView	a scrolling grid of tiles (a photo gallery)
SingleChildScrollView	makes <i>one</i> big child scroll: a form, a page of text
PageView	pages that you swipe, one after the other
CustomScrollView	slivers: an app bar that collapses, several lists in one scroll

11 Going further

The two steps below are not in the slides. They add what almost every form needs.

Reading and changing the text: `TextEditingController`

`onChanged` tells you about the text when the *user* changes it. Sometimes the *program* must read the text at any moment, or change it: to clear a field after “Send”, to give it a first value, to change its case. For this we give the `TextField` a `TextEditingController`. It holds the text: `controller.text` reads it, `controller.text = 'abc'` changes it, and `controller.clear()` empties it.

A controller must be released when the widget goes away. We create it as a field of the state class and we release it in the `dispose` method (the end of the life of the state object, figure 1):

```
1 class _ControllerDemoState extends State<ControllerDemo> {
2   // The controller holds the text of the field.
3   final TextEditingController _controller =
4     TextEditingController(text: 'Flutter');
5
6   @override
7   void dispose() {
8     _controller.dispose(); // release the controller
9     super.dispose();
10  }
```

- `TextField(controller: _controller)` connects them. Example `06_text_controller` starts with the text `Flutter` in the field, and has buttons that change it.
- When the *program* changes the text, `onChanged` is **not** called. If something else on the screen depends on the text, call `setState` yourself, as the **CLEAR** button does.

Reading numbers safely

A `TextField` always gives a `String`. The user can type anything, so the conversion must not crash:

Code	Result for 'abc'
<code>double.parse('abc')</code>	stops the app: <code>FormatException</code>
<code>double.tryParse('abc')</code>	<code>null</code> : no crash

Always use `tryParse` for text typed by a user, then test the result for `null` (week 1, null safety). Example `08_safe_number_input` goes one step further and *helps* the user. First, a numeric keyboard on a phone, with a decimal point and a minus sign:

```
1 keyboardType: TextInputType.numberWithOptions(decimal: true, signed: true),
```

Second, `errorText` shows a red message under a field that holds something that is not a number, and nothing (`null`) when the field is empty or valid:

```
1 // The message under a field: only when something was typed and it is wrong.
2 String? _errorOf(String text) {
3   if (text.trim().isEmpty) return null;
4   return double.tryParse(text.trim()) == null ? 'Not a number' : null;
5 }
```

Third, the **SUM** button is disabled (`onPressed: null`) until both fields hold a valid number: the user sees at once that the button cannot be used yet.

```

1 ElevatedButton(
2   onPressed: ready ? _sum : null, // null: disabled
3   child: const Text('SUM', style: TextStyle(fontSize: 24)),
4 ),

```

12 Common errors

Message or symptom	Cause	Fix
The screen does not change	the variable was changed without <code>setState</code>	change it inside <code>setState</code>
<i>setState() or markNeedsBuild() called during build</i>	<code>onPressed:</code> <code>updateText()</code> : the function is <i>called</i> while the page is built	remove the parentheses: <code>onPressed: updateText,</code> or write <code>() { updateText(); }</code>
<i>setState() called after dispose()</i>	<code>setState</code> runs after the widget has left the screen (for example after a delay)	test if <code>(!mounted)</code> <code>return;</code> before <code>setState</code>
<i>No Material widget found</i>	a <code>TextField</code> outside a <code>Scaffold</code> or a <code>Material</code>	put the page in a <code>Scaffold</code>
<i>An InputDecorator ... cannot have an unbounded width</i>	a <code>TextField</code> directly in a <code>Row</code>	wrap it in <code>Expanded</code> or <code>SizedBox(width: ...)</code>
<i>A RenderFlex overflowed by ...pixels on the bottom</i>	the keyboard takes the space and the content no longer fits	put the body in a <code>SingleChildScrollView</code>
<i>FormatException</i>	<code>double.parse</code> or <code>int.parse</code> on a text that is not a number	use <code>tryParse</code> and test for <code>null</code>

13 Exercises

- In example 03_you_typed, remove `setState` from `updateText` (keep the assignment). What happens? Put it back.
- In 04_custom_textfield, change the font size and the border of `MyTextField`, in one place only. Add a second `MyTextField` to the page and check that it has the same design.
- Add an optional parameter `onSubmitted` to `MyTextField` (type `ValueChanged<String>?`) and pass it to the `TextField`. Use it to display “Done: ...” when the user presses Enter.
- In the sum app, make the **SUM** button round (a `StadiumBorder`), then a circle with an icon.
- Add a third field *Z* to the sum app, and display $X + Y + Z$.
- Add buttons $X \times Y$ and $X \div Y$. What must the division do when *Y* is 0? Display a message.
- Use `TextEditingController` in the sum app, and add a **CLEAR** button that empties both fields and the result.
- Make a password field: `obscureText: true`, and a `Switch` “Show password” that turns `obscureText` off. What must be stateful?
- Move the class `MyTextField` of example 04_custom_textfield to its own file and import it, as in 05_sum_app.

10. In `09_lifecycle_log`, add a `didChangeDependencies` override to the parent page too, and see when it is called compared to the child's.
11. In `12_stopwatch_laps`, show the *fastest* lap (the smallest value in `_laps`) above the list.
12. Change `10_stopwatch_timer` so the interval is 500 milliseconds instead of one second, and update `_secondsText` to show tenths of a second.
13. In `13_scrolling`, replace `ListView.builder` by `ListView.separated` and add a `Divider` between the rows.
14. In `13_scrolling`, remove the `Expanded` around the `ListView` and read the error in the Debug Console.

14 Summary

Idea	In one sentence
<code>StatefulWidget</code>	an immutable widget class whose only job is <code>createState()</code>
<code>State</code>	keeps the data that changes and has <code>build</code> ; private (<code>_Name</code>)
<code>setState</code>	changes the data and tells Flutter to call <code>build</code> again; without it the screen does not change
Life cycle	<code>initState</code> (once, start), <code>build</code> (required, every rebuild), <code>dispose</code> (once, release resources); also <code>didChangeDependencies</code> and <code>didUpdateWidget</code>
<code>TextField</code>	takes text from the user: <code>style</code> , <code>decoration</code> , <code>onChanged</code> , <code>onSubmitted</code> , <code>keyboardType</code> , <code>controller</code>
<code>InputDecoration</code>	<code>border</code> , <code>hintText</code> , <code>labelText</code> , <code>icons</code> , <code>errorText</code>
<code>SizedBox</code>	gives its child a width and a height
<code>callback</code>	a function given to a widget to be called later: <code>onChanged</code> : <code>updateText</code> (no parentheses)
<code>ValueChanged<T></code>	the type of a function that takes one value of type <code>T</code>
<code>library</code> / <code>import</code>	every file is a library; <code>import 'home.dart'</code> ; is relative, <code>package:</code> is for packages; <code>_</code> means private to the file
<code>ElevatedButton</code>	<code>child</code> , <code>onPressed</code> (null disables it), <code>style</code> with <code>styleFrom</code> and <code>shape</code>
<code>Timer.periodic</code>	calls a function again and again; cancel it in <code>dispose</code>
<code>ListView</code> / <code>.builder</code>	a scrolling list of widgets; <code>.builder</code> builds only the visible rows
<code>Expanded</code>	needed around a scrolling widget inside a <code>Row</code> or <code>Column</code>
<code>TextEditingController</code>	reads and changes the text of a field; released in <code>dispose</code>
<code>double.tryParse</code>	converts a text to a number and returns <code>null</code> instead of crashing

This document is maintained by Dr. Mohamad AOUE

Last updated: 2026-09-22